STM32 + 树莓派5 双电机与LED控制说明

1. 仓库用途

这个仓库原本是一个 STM32F103 + TB6612 的双路电机控制示例：

- STM32 通过串口接收上位机指令
- 根据指令设置两路电机方向和 PWM 占空比
- 适配 Keil 工程（`stm32/USER/Tb6612demo.uvprojx`）

本次改造后，系统支持：

- STM32 接收 JSON 控制命令
- 增加 LED 状态机控制
- STM32 向树莓派返回 JSON 成功/失败应答
- 树莓派5 使用 C++ 控制程序，支持交互输入 / 命令行参数 / JSON 文件

2. 硬件连接

2.1 TB6612 与 STM32

现有电机控制引脚（沿用原工程）：

- 电机1方向：PB7 / PB8
- 电机1 PWM：PB6 (TIM4_CH1)
- 电机2方向：PA1 / PA2
- 电机2 PWM：PA6 (TIM3_CH1)

2.2 串口连接（树莓派5 <-> STM32）

- STM32 USART1_TX (PA9) -> 树莓派 RX
- STM32 USART1_RX (PA10) -> 树莓派 TX
- GND 必须共地
- 波特率默认 115200，8N1

注意：如果电平不匹配，请使用合适的电平转换模块。

2.3 LED 连接

当前实现使用 PC13 作为 LED 引脚（低电平点亮）。

- 若是 Blue Pill 板载 LED：通常就是 PC13，直接使用
- 若是外接 LED：
  - PC13 -> 电阻 -> LED -> 电源/地（按低电平点亮逻辑接线）
  - 或改代码中的 LED 引脚宏到你的实际 IO

3. LED 亮灯逻辑

STM32 上电后 LED 逻辑如下：

1. 上电常亮 5 秒
2. 随后闪烁 3 次
3. 闪烁结束后熄灭

运行中：

- 串口发生数据交互时，LED 快速闪烁一次作为通信指示
- 非交互时刻，LED 由控制程序中的 `led_idle` 字段决定（空闲亮或灭）

4. STM32 JSON 协议

STM32 按“**一行一条 JSON，CRLF 结尾**”接收指令。

支持字段：

- `led_idle`: `0/1` 或 `false/true`
- `m1_dir`: `"forward"`/`"reverse"` 或 `0/1`
- `m1_speed`: `0~3000`
- `m1_duration_ms`: 电机1运行时长（毫秒，>0生效）
- `m2_dir`: `"forward"`/`"reverse"` 或 `0/1`
- `m2_speed`: `0~3000`
- `m2_duration_ms`: 电机2运行时长（毫秒，>0生效）

示例：

```
{"led_idle":1,"m1_dir":"forward","m1_speed":1000,"m1_duration_ms":1200,"m2_dir":"reverse","m2_speed":800,"m2_duration_ms":900}
```

STM32 返回示例：

成功：

```
{"ok":true,"code":"ok","message_cn":"执行成功","message_en":"success"}
```

失败（例如 JSON 未解析到有效字段）：

```
{"ok":false,"code":"parse_error","message_cn":"JSON解析失败","message_en":"json parse failed"}
```

5. 树莓派5 C++ 控制程序

路径：`rpi_controller/`

主程序：`rpi_controller/src/main.cpp`

特性：

- Linux 端使用 POSIX 串口
- Windows 端使用 Windows API 串口
- 三种交互模式：
  - 启动后逐行输入
  - 命令行参数
  - JSON 文件
- 中英双语输出
- 自动日志：运行目录下 `logs/时间戳.log`

6. 使用方法

6.1 本机构建（示例）

```
cmake -S rpi_controller -B rpi_controller/build-native -
DCMAKE_BUILD_TYPE=Release
cmake --build rpi_controller/build-native -j
```

6.2 交互模式

```
./rpi_controller/build-native/rpi5_stm32_controller --port /dev/ttyUSB0 --
interactive
```

交互输入示例：

```
led=on m1_dir=forward m1_speed=1200 m1_time=1500
m2_dir=reverse m2_speed=900 m2_time=1000
```

也可直接输入 JSON：

```
{"led_idle":1,"m1_dir":"forward","m1_speed":1000,"m1_duration_ms":1000}
```

6.3 命令行参数模式

```
./rpi_controller/build-native/rpi5_stm32_controller \
  --port /dev/ttyUSB0 \
  --led-idle on \
  --m1-dir forward --m1-speed 1200 --m1-time 1500 \
  --m2-dir reverse --m2-speed 800 --m2-time 1000
```

6.4 JSON 文件模式

```
./rpi_controller/build-native/rpi5_stm32_controller \
  --port /dev/ttyUSB0 \
  --json-file rpi_controller/examples/command_example.json
```

7. 交叉编译

已提供脚本：

- `rpi_controller/scripts/build_linux.sh`
- `rpi_controller/scripts/build_windows.sh`

执行：

```
bash rpi_controller/scripts/build_linux.sh
bash rpi_controller/scripts/build_windows.sh
```

说明：

- Linux 交叉编译脚本默认使用 `x86_64-linux-gnu-g++`
- Windows 交叉编译脚本默认使用 `x86_64-w64-mingw32-g++`
- 若本机未安装对应工具链，脚本会提示缺失并退出

8. 关键源码位置

- STM32 主控制逻辑：`stm32/USER/main.c`
- LED GPIO 接口：`stm32/HAREWER/GPIO/gpio.c`、`stm32/HAREWER/GPIO/gpio.h`
- 树莓派控制程序：`rpi_controller/src/main.cpp`